

obstacles in the construction of practical quantum computers. In other words, the biggest challenge is not creating a qubit, but keeping it ‘quantum’ long enough to perform meaningful computations.

To overcome these challenges, engineers employ some of the most sophisticated technologies available today. Superconducting qubits are fabricated using advanced nanofabrication techniques and operated inside dilution refrigerators that cool them to temperatures of only a few millikelvin—colder than outer space. Precisely shaped microwave pulses are then used to manipulate individual qubits and implement quantum gates, while elaborate shielding and filtering systems protect them from external noise. Even after all these engineering efforts, present-day quantum processors remain noisy, giving rise to the term Noisy Intermediate-Scale Quantum (NISQ) computers. This explains why running the same quantum program on a simulator and on a real quantum processor often produces noticeably different results.

Now that we understand that present-day quantum hardware is not perfectly reliable, an obvious question arises: how can we imitate the error-prone behaviour of a real quantum computer while developing and testing quantum algorithms? Qiskit provides us with methods to simulate such noisy quantum systems. To understand this, let us revisit the program *qiskit4.py*, discussed in the second article of this series, which demonstrated the operation of the Pauli-X gate. You may recall that after executing the circuit for 1000 shots, the output was always $\{1: 1000\}$. This is because the Pauli-X gate behaves like the classical NOT gate, flipping the initial qubit state from $|0\rangle$ to $|1\rangle$ every time it is applied. Since the qubit is measured in a computational basis, the state $|1\rangle$ always produces the classical output 1. The result clearly demonstrates that the simulator assumes an ideal quantum

computer, where every gate and every measurement is performed perfectly without any influence of hardware-induced errors.

Now, consider the program *qiskit7.py* (with line numbers included for clarity), shown below, which artificially introduces errors into the quantum system.

```
1. from qiskit import QuantumCircuit, transpile
2. from qiskit_aer import AerSimulator
3. from qiskit_aer.noise import NoiseModel, pauli_error

4. qc = QuantumCircuit(1, 1)
5. qc.x(0)
6. qc.measure(0, 0)

7. noise_model = NoiseModel( )
8. error = pauli_error([("X", 0.05), ("I", 0.95)])
9. noise_model.add_all_qubit_quantum_error(error, ["x"])
10. noisy_sim = AerSimulator(noise_model=noise_model)
11. compiled = transpile(qc, noisy_sim)
12. result = noisy_sim.run(compiled, shots=1000).result( )
13. print(result.get_counts( ))
```

In the program above, the statements used to create the quantum circuit, apply the Pauli-X gate, perform measurements, and execute the circuit on the simulator were explained previously. Therefore, the following discussion focuses only on the newly introduced statements that allow the simulator to model the behaviour of a noisy quantum computer. In addition to importing the class *QuantumCircuit*, Line 1 imports the function *transpile()*. Transpilation is the process of converting a quantum circuit into a form that is compatible with a specific quantum backend. Since we shall be executing the circuit on a noisy simulator configured to imitate a real quantum processor,

the circuit must first be transpiled for that backend. Line 2 imports the class *AerSimulator*. Line 3 imports the class *NoiseModel* and the function *pauli_error()* from Qiskit’s *noise* module. The class *NoiseModel* is used to define a noise model that describes how errors are introduced into the quantum circuit during simulation. The helper function *pauli_error()* creates an error channel based on Pauli operators. In this example, it is used to introduce errors while executing the Pauli-X gate. Lines 4 to 6 create a single-qubit circuit, apply the Pauli-X gate, and measure the qubit.

Line 7 creates an empty noise model. At this stage, the model does not contain any errors. Line 8 defines a Pauli error model. The list supplied to the function *pauli_error()* specifies the probabilities associated with different operations: (“X”, 0.05) means that an unwanted Pauli-X error is introduced with a probability of 5% and (“I”, 0.95) means that with a probability of 95%, the identity operation is applied, indicating that no error occurs. Thus, the simulator behaves as though every execution of the gate has a 5% chance of suffering an unwanted Pauli-X error. Line 9 tells the noise model to associate the previously defined error with every Pauli-X gate in the circuit. Consequently, whenever a Pauli-X gate is executed during the simulation, the specified error model is applied automatically. Instead of creating an ideal simulator, Line 10 creates an *AerSimulator* configured with the noise model defined earlier. As a result, the simulator attempts to imitate the behaviour of a noisy superconducting quantum computer rather than an ideal one. Line 11 transpiles the circuit for a noisy simulator before execution. This step ensures that the circuit is compatible with the target backend and allows the simulator to apply the specified noise model correctly. Line 12 executes the transpiled circuit on the noisy simulator for 1,000